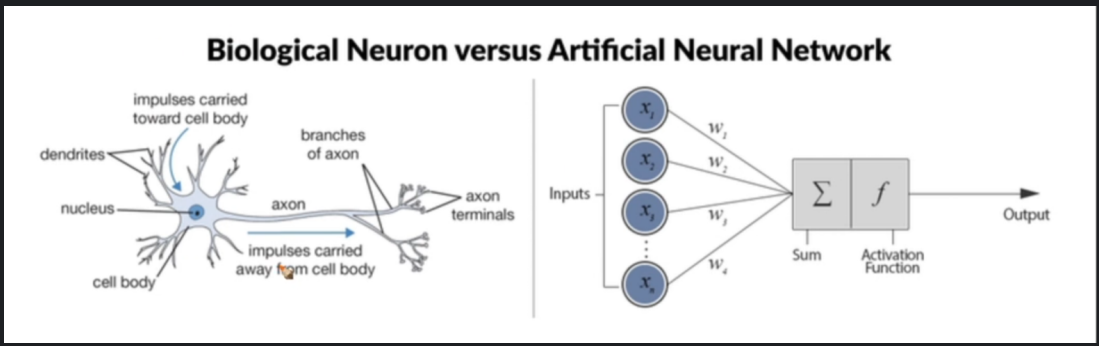
॥ श्रीगणेशाय नमः ॥

Deep Learning: Neural Networks

History / Introduction

In 1943, Warren McCulloch and Walter Pitts invented the Perceptron. Its first hardware implementation was built by Frank Rosenblatt in 1957.
Artificial neurons take inspiration from how biological neurons work.



In a very basic sense, the biological neuron receives electric impulses via its dendrites. It does some kind of computation inside, and forwards the signal to other neurons.
In an artificial neuron, the inputs are given as vectors, some given more weights than others. These are summed and subjected to an activation function that gives an output to be forwarded ahead.

In Biology, a neuron didn't exist on its own, but as a network of neurons connected to each other by synapses. A similar structure was created (neural network).
The first deep learning MLP was published by Ivakhnenko and Lapa in 1965. The first deep learning MLP trained by stochastic gradient descent was published in 1967 by Shun'ichi Amari.
In 1986, David E. Rumelhart and Geoffrey Hinton published an experimental analysis of backpropogation.

The first AI winter saw a downfall of interest in neural networks.
Many techniques evolved around 1995-2009.

Geoffrey Hinton in 2006 came up with a technique to train neural networks.
In 2012, in ImageNet, using a lot of computation, a deep neural net was built to beat every other algo by a huge margin. That sparked a huge interest in deep learning.

Since then, there's been an exponential increase in the amount of data, amount of computation, and a rise of newer algorithms.

How biological neurons work

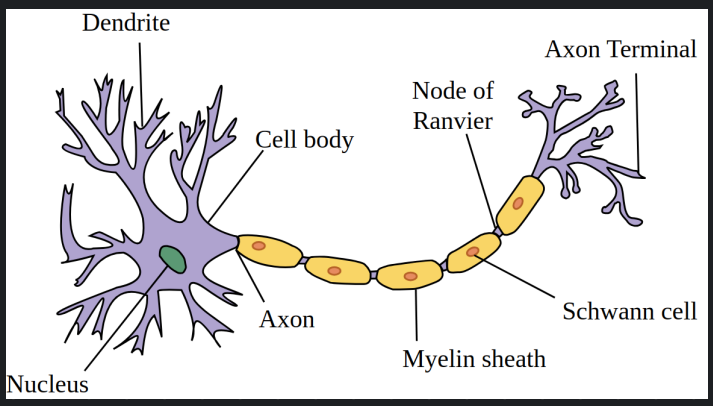
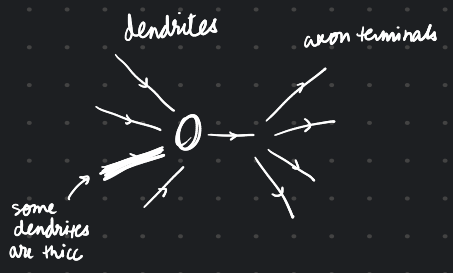
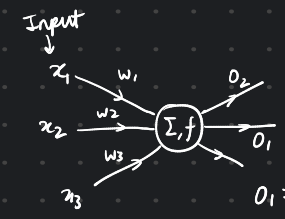


Diagram of the components of a neuron

- The **soma** is the body of the neuron. As it contains the nucleus, most protein synthesis occurs here. The nucleus can range from 3 to 18 micrometers in diameter.[7]
- The **dendrites** of a neuron are cellular extensions with many branches. This overall shape and structure are referred to metaphorically as a dendritic tree. This is where the majority of input to the neuron occurs via the dendritic spine.
- The **axon** is a finer, cable-like projection that can extend tens, hundreds, or even tens of thousands of times the diameter of the soma in length. The axon primarily carries nerve signals away from the soma and carries some types of information back to it. Many neurons have only one axon, but this axon may—and usually will—undergo extensive branching, enabling communication with many target cells. The part of the axon where it emerges from the soma is called the **axon hillock**. Besides being an anatomical structure, the axon hillock also has the greatest density of voltage-dependent sodium channels. This makes it the most easily excited part of the neuron and the spike initiation zone for the axon. In electrophysiological terms, it has the most negative threshold potential.
  - While the axon and axon hillock are generally involved in information outflow, this region can also receive input from other neurons.
- The **axon terminal** is found at the end of the axon farthest from the soma and contains synapses. Synaptic boutons are specialized structures where neurotransmitter chemicals are released to communicate with target neurons. In addition to synaptic boutons at the axon terminal, a neuron may have *en passant* boutons, which are located along the length of the axon.



A neuron is activated/fired when there's enough input signals.



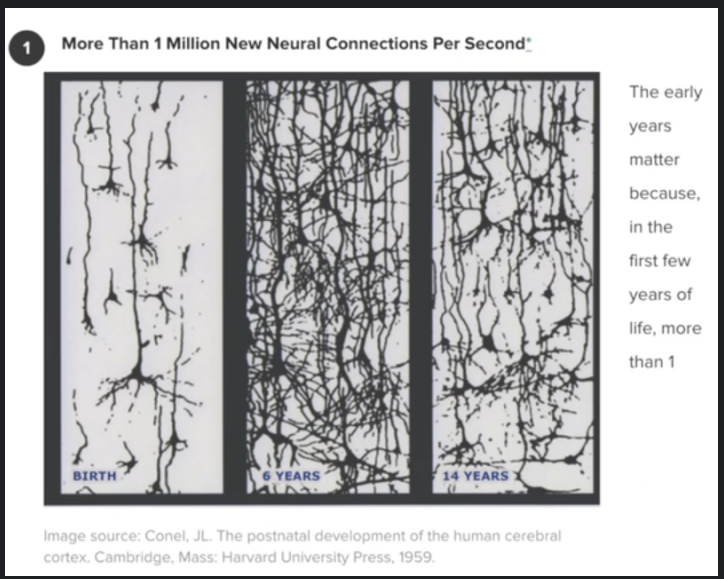
inputs are given weights

$o_1 = f(w_1 x_1 + w_2 x_2 + \dots)$

$o = f\left(\sum_{i=1}^{n} w_i x_i\right)$

output — activation func — weights — inputs

Biological neural networks



NN is sparse at birth. (think of it as weights betⁿ many neurons are 0)
As the age increases,
the neural network grows denser as the child learns.

Learning is just forming more connections betⁿ neurons.
↳ done via input data (visual, auditory, sensory)
Eventually, useless connections are broken since it takes resources to sustain those.
Around 80, neural degeneration takes place.

All biological learning is just adjusting wts. on neurons.

Logistic regression and perceptron

Logistic regression

$x_i \rightarrow \hat{y}_i \quad D = \{x_i, y_i\} \quad x_i \in \mathbb{R}^d$

$\hat{y}_i = \text{sigmoid}(w^T x_i + b) \quad w \in \mathbb{R}^d, b \in \mathbb{R}$

$= \text{sigmoid}\left(\sum_{j=1}^{d} w_j x_{ij} + b\right)$

$x_i = [x_{i1}, x_{i2}, x_{i3}, \ldots, x_{id}] \quad W = [w_1, w_2, \ldots, w_d]$

In a neuron, $O = f\left(\sum_{j=1}^{d} w_j x_{ij}\right)$

logistic regression can be represented using a neuron:



$\hat{y}_i = f(w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + 1 \cdot b)$

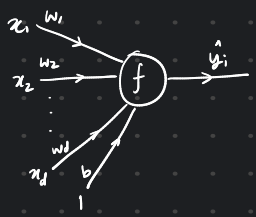
$= f(w^T x_i + b)$

$\hat{y}_i = \text{Sigmoid}(w^T x_i + b)$

we used SGD to train LR.

Training a NN: Finding weights on the edges.

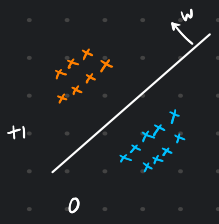
Perceptron



activn func^

$$f(x) = \begin{cases} 1 & \text{if } w^T x > 0 \rightarrow \text{Neuron fires/activates} \\ 0 & \text{otherwise} \rightarrow \text{doesn't fire} \end{cases}$$

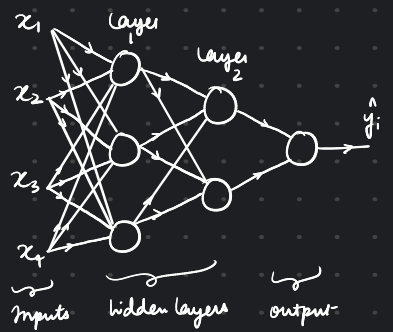
A perceptron is a linear classifier.



No squashing in a basic perceptron.

Multi-layered perceptron (MLP)

An MLP, as opposed a perceptron which has a single neuron, has many neurons connected to form a network (NN).



Why care about these?

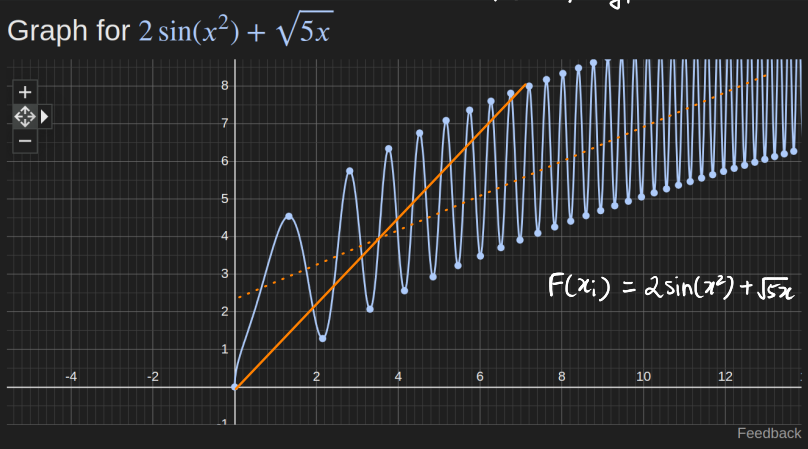
Consider a regression problem.

$D = \{x_i, y_i\}$

$\hat{y}_i = f(x_i)$

$x_i \in \mathbb{R}^1, \; y_i \in \mathbb{R}$

Task : to find $F(x_i) = y_i$

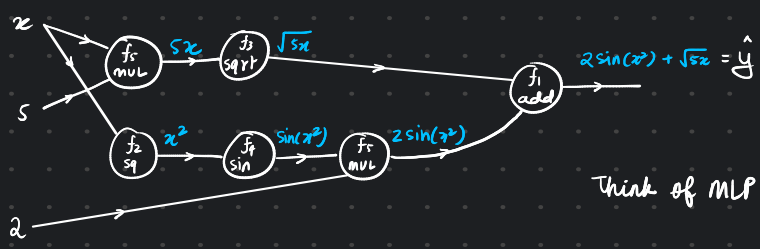


case 1 : $F(x_i) = x_i = y_i$

Too simple, lots of errors.
Even if you consider a better line, there'll be a lot of errors.

The function F is quite complex. It is a composite of simpler functions.
Say we represent it using those simpler functions.
We could have something like an MLP.

$f_1 \rightarrow$ add( )
$f_2 \rightarrow$ square( )
$f_3 \rightarrow$ sqrt( )
$f_4 \rightarrow$ sin( )
$f_5 \rightarrow$ multiply( )



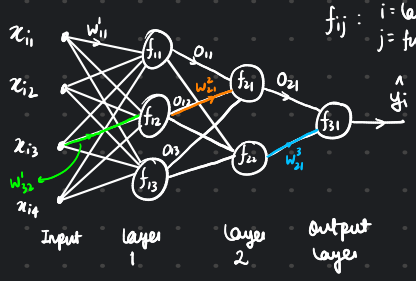
Think of MLP as graphical composition.

$$F(x) = f_1\left(f_3(f_5(x, 5)), f_5(f_4(f_2(x)), 2)\right)$$

Essentially we create non-linearity using multiple layers to solve harder problems.
Multi-layered models result in really powerful models.
But this also means easy overfitting.

Notation

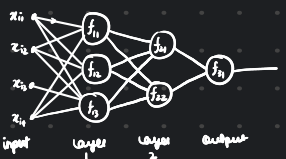
Say $x_i \in \mathbb{R}^4$, $y_i \in \mathbb{R}$



f_{21} : 2nd layer, func° #1.
f_{ij} : i = layer #
j = func° index.

O_{ij} : output of f_{ij} (i^{th} layer, j^{th} neuron)

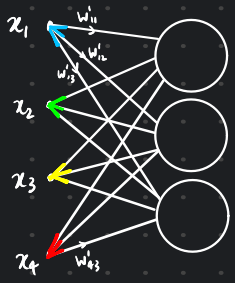
W^k_{ij} : { k : next layer, i : from, j : to





Here we make all the possible connections between 2 layers of neurons.
This is called a fully-connected NN or MLP.

Consider the first layer.



$4 \times 3 = 12$ wts. in total.

$$W^1_{[4\times3]} = \begin{bmatrix} W^1_{11} & W^1_{12} & W^1_{13} \\ W^1_{21} & W^1_{22} & W^1_{23} \\ W^1_{31} & W^1_{32} & W^1_{33} \\ W^1_{41} & W^1_{42} & W^1_{43} \end{bmatrix}$$

Ily there's $W^2_{[3\times2]}$ and $W^3_{[2\times1]}$

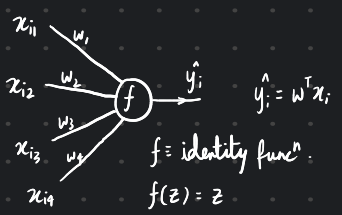
Training a single neuron model

Here training means to find the best edge weights using the training data.

Consider the linear regression problem represented as a neuron model.

$\hat{y}_i = \sum_{j=1}^{d} w_j x_{ij}$, $D = \{x_i, y_i\}$, $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$. Task: find the optimal w.

$= w^T x_i$



$\hat{y}_i = w^T x_i$

f = identity func^n.

$f(z) = z$

We represent it as an optimiza^n problem:

$$\min_{w_i} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \text{regularizer}$$

$$\Rightarrow \min_{w_i} \sum_{i=1}^{n} (y_i - w^T x_i)^2 + \|w\|_2^2 \longrightarrow \text{prev. solved using SGD.}$$

① We define a loss func^n.

Loss func^n $L = \sum_{i=1}^{n} \underbrace{(y_i - \hat{y}_i)^2}_{L_i} + \text{reg}$

$L = \sum_{i=1}^{n} L_i$

② Optimiza^n problem.

$$\min_{w_i} \sum_{i=1}^{n} \underbrace{(y_i - w^T x_i)^2}_{\hat{y}_i = f(w^T x_i)}$$

more general

$$w^* = \arg\min_{w_i} \sum_{i=1}^{n} (y_i - f(w^T x_i))^2 + \text{reg}$$

f < identity : lin. reg.
sigmoid : log. reg.
threshold : perceptron

③ solve the optimiza^n problem.

(SGD)

a) Initializa^n of weights (w_i)

b) $\nabla_w L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial w_4} \end{bmatrix}$

$= \begin{bmatrix} \frac{\partial L}{\partial w_1} & \frac{\partial L}{\partial w_2} & \frac{\partial L}{\partial w_3} & \frac{\partial L}{\partial w_3} \end{bmatrix}^T$

c) $W_{new} = W_{old} - \eta \nabla_w L$

or $(w_i)_{new} = (w_i)_{old} - \eta \left[\frac{\partial L}{\partial w_i}\right]_{(w_i)_{old}}$

for iter = 1 to k

$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial f} \cdot \frac{\partial f}{\partial w_1}$ → chain rule

$L = \sum_{i=1}^{n} (y_i - f(w^T x_i))^2$

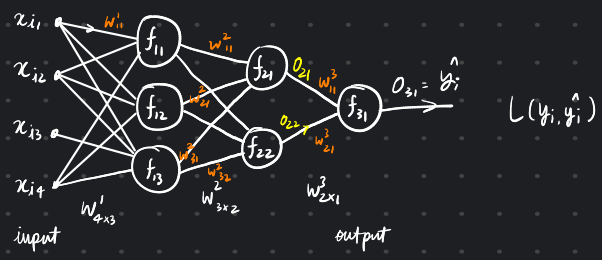
$\frac{\partial L}{\partial f} = -2 \sum_{i=1}^{n} (y_i - f(w^T x_i))$; $\frac{\partial f}{\partial w_1} = x_{i1}$

$\Rightarrow \frac{\partial L_i}{\partial w_1} = -2 x_{i1} (y_i - \hat{y}_i)$

$\Rightarrow \frac{\partial L}{\partial w_1} = -2 \sum_{i=1}^{n} x_{i1} (y_i - \hat{y}_i)$

Training an MLP: chain rule

$D = \{x_i, y_i\}$ $x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$. We'll use the sqd. loss.



At the end of training, we'd like to determine $W^1_{4\times3}$, $W^2_{3\times2}$, $W^3_{2\times1}$ = 20 weights. Training = determining weights.

step ① Define a loss func^n:

$$L = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \text{reg} \qquad \text{reg} \to \sum_{i,j,k} (W^k_{ij})^2 \text{ (sum like L2 regularizn)} \quad \text{or} \quad \sum_{i,j,k} |W^k_{ij}|$$

$$L_i = (y_i - \hat{y}_i)^2$$

$$L = \sum_{i=1}^{n} L_i + \text{reg}$$

Optimizn problem: $\min_{W^1, W^2, W^3} L$ → sqd. loss, regularizn | $\min_{W^k_{ij}} L$

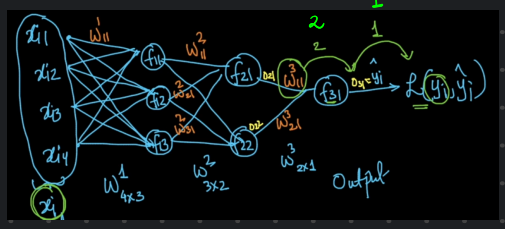
② Gradient descent: $\frac{\partial L}{\partial W^k_{ij}}$

a) Initialize W^k_{ij} randomly.

b) $(W^k_{ij})_{new} = (W^k_{ij})_{old} - \eta \boxed{\frac{\partial L}{\partial W^k_{ij}}}$ ← update rule.

η ↑ learning rate

③ Perform updates till convergence.



e.g. consider $\frac{\partial L}{\partial W^3_{11}}$

W^3:

$$\frac{\partial L}{\partial W^3_{11}} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial W^3_{11}}$$

1 2

$$\frac{\partial L}{\partial W^3_{21}} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial W^3_{21}}$$

W^2:

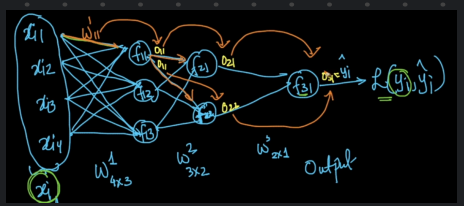
$$\frac{\partial L}{\partial W^2_{11}} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W^2_{11}}$$

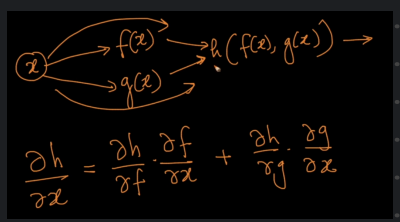
$$\frac{\partial L}{\partial W^2_{21}} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W^2_{21}}$$

$$\frac{\partial L}{\partial W^2_{31}} = \frac{\partial L}{\partial O_{31}} \cdot \frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial W^2_{31}}$$ and so on.

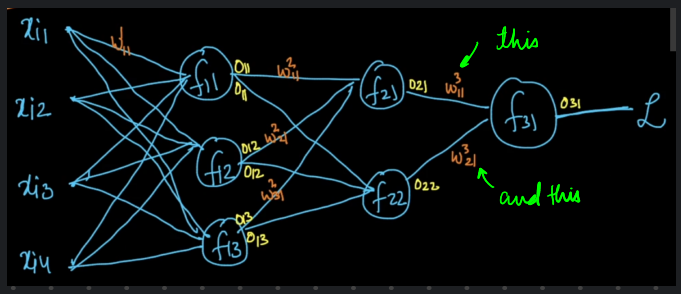
W^1:

$$\frac{\partial L}{\partial W^1_{11}} = \frac{\partial L}{\partial O_{31}} \left[\frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W^1_{11}} + \frac{\partial O_{31}}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W^1_{11}} \right]$$





Training an MLP: memoization



Say we're calculating these:

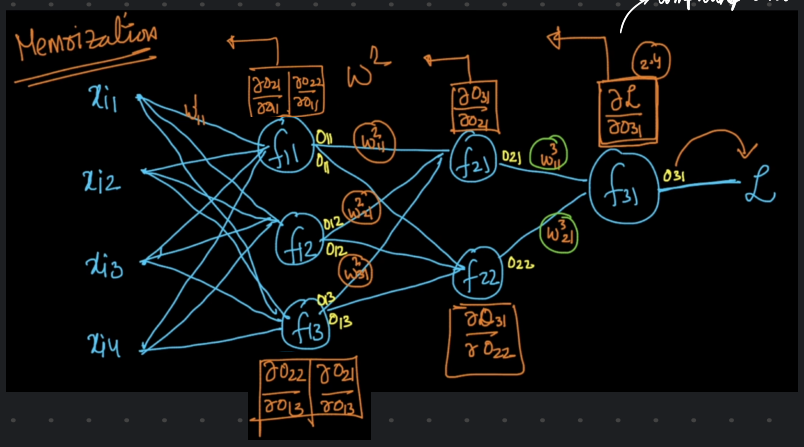
$$\frac{\partial L}{\partial w^3_{11}} = \frac{\partial L}{\partial o_{31}} \cdot \frac{\partial o_{31}}{\partial w^3_{11}}$$

↑ we're calculating this twice. We shouldn't need to.

$$\frac{\partial L}{\partial w^3_{21}} = \frac{\partial L}{\partial o_{31}} \cdot \frac{\partial o_{31}}{\partial w^3_{21}}$$

The idea is that we store all these repeated values once and reuse them. Computing the derivatives is not really trivial. So this saves a lot of time.

If there's any operation that's used repeatedly, it is a good idea to compute it once, store it, and reuse it.



For slightly more used memory, we get a lot of speed-up.

Backpropogation = chain rule + memoization

Backpropogation algorithm

$D = \{x_i, y_i\}_{i=1}^{n}$

① Initialize w_{ij}^{k}'s.

② for each x_i in D:

- ⓐ pass x_i forward through the network. → forward propogation
- ⓑ compute $L(\hat{y_i}, y_i)$
- ⓒ compute all derivatives you need using chain-rule and memoizaⁿ.
- ⓓ update weights from end of the network to the start. → Back propogation

$$(w_{11}^{3})_{new} = (w_{11}^{3})_{old} + \eta \frac{\partial L}{\partial w_{11}^{3}} \quad \left(\frac{\partial L}{\partial w_{11}^{3}} = \frac{\partial L}{\partial o_{31}} \cdot \frac{\partial o_{31}}{\partial w_{11}^{3}}\right)$$

← gradient update

so what we're doing is sending input to get output $\hat{y_i}$.
Then we're using error to update weights so that the next time we send inputs, we get lesser error.

③ Repeat ② till convergence.

Epoch: 1 epoch means we've passed all the points in D once to the network. We run a model for multiple epochs.

Algo summary:
1. Initialization
2. Epoch:
    - forward prop
    - compute loss
    - compute derivatives (chain-rule and memoization)
    - update weights (backward prop)
3. Repeat

Backpropogation works only if the activation functions are differentiable (We get outputs O_ij from the functions).
A function that is easily/quickly differentiable speeds up the training of a neural network.

$x_i \xrightarrow[w^1]{} \text{layer 1} \xrightarrow[w^2]{} \text{layer 2} \xrightarrow[w^3]{} \hat{y_i}$

Here, we send each point into the neural network.
In SGD, we use 1 point. In mini-batch SGD, we use a set of points. In GD, we use all the points in D.

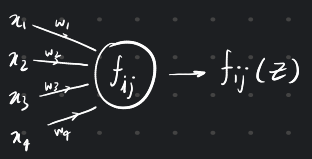
An idea is that instead like in SGD (sending a single point),
we use mini-batch back-propogation, where instead of sending in a point, we send a subset of points $S \subseteq D$.
$|S|$ = mini-batch size.

A problem is keeping all the points in memory.

Mini-batch back propogation is a really popular approach.
Say we have 10,000 points and we choose a mini-batch size of 100 (usually powers of 2, considering how it leverages the RAM).
For 1 epoch, we run 10,000/100 = 100 loops (each for 1 batch, instead of 1 point).
So what would've cost us 10,000 runs would cost us 100 runs using mini-batch back-propogation.

Activation functions

An activation function should be differentiable and easy to differentiate.



$X^T = [x_1\ x_2\ x_3\ x_4]$

$W^T = [w_1\ w_2\ w_3\ w_4]$

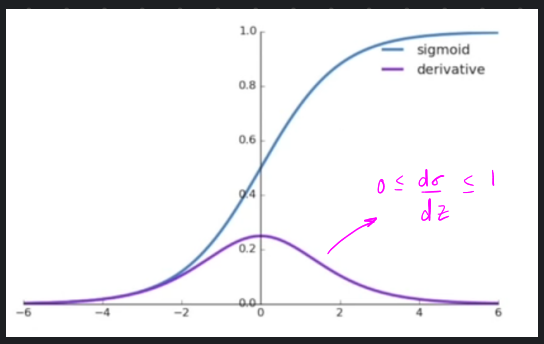
$$z = \sum_i w_i x_i = W^T X$$

Sigmoid

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

The derivative of the sigmoid can be represented in terms of the sigmoid itself.
One implementation of the sigmoid could be repurposed for forward as well as back-propogation.

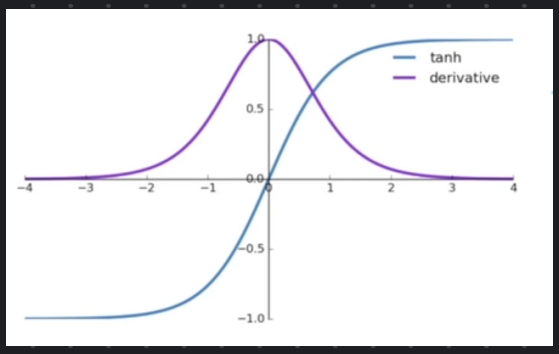
$$\frac{\partial \sigma}{\partial z} = \sigma(z)(1-\sigma(z))$$



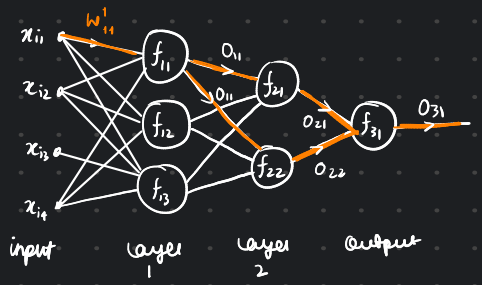
tanh

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\frac{d}{dz}(\tanh) = 1 - \tanh^2(z)$$



Vanishing gradient problem



Mainly, the activation functions used as the f_i's were sigmoid or tanh.

$$(W'_{11})_{new} = (W'_{11})_{old} - \eta \frac{\partial L}{\partial W'_{11}}$$

$$\frac{\partial L}{\partial W'_{11}} = \frac{\partial L}{\partial O_{31}} \left[\frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} + \frac{\partial O_{31}}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} \right]$$

$O_{31} \rightarrow$ output of f_{31}

$$\frac{\partial O_{31}}{\partial O_{21}} = \frac{\partial f_{31}}{\partial O_{21}}$$

And f_{31} is sigmoid,
so $\frac{\partial O_{31}}{\partial O_{21}}$ is in $[0, 1)$

$$\frac{\partial L}{\partial O_{31}} \left[\frac{\partial O_{31}}{\partial O_{21}} \cdot \frac{\partial O_{21}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} + \frac{\partial O_{31}}{\partial O_{22}} \cdot \frac{\partial O_{22}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W'_{11}} \right]$$

$0 \leq$ < 1 $[0,1)$ $[0,1)$

[really small as it is the sigmoid function]

In case of sigmoid the derivative < 0.3.
Say they are $0.2, 0.1, 0.05$
product $= 0.0010 = 10^{-3} \ll 1$.

We are using such small values in our update function.

$$(W'_{11})_{new} = (W'_{11})_{old} - \eta \frac{\partial L}{\partial W'_{11}}$$

Say $= 2.5$ | Say $\eta = 1$ | $\frac{\partial L}{\partial W'_{11}}$ really small, say 0.001

Then $(W'_{11})_{new} = 2.5 - 1 \times 0.001 = 2.499$

$$(W'_{11})_{new} \approx (W'_{11})_{old}$$

So our main problem is that the updated weights are too close to the old weights, because the derivative of loss w.r.t. weight is small (because this is a product of derivatives each < 1, and these are small because of sigmoid and tanh).
In other words, sigmoid and tanh makes weight updation really hard.
This gets worse when there are more hidden layers as the # of multiplications = # of hidden layers + 1.

so if there are like 10 hidden layers, the value of $\frac{\partial L}{\partial w}$ would be like 10^{-15} or something.

so there would be NO difference in W_{new} and W_{old}.

$W_{new} = W_{old}$

The final gradient that we get, $\delta L / \delta w$, it just vanishes. The main problem is due to the sigmoid/tanh activation functions.
This prevented training of deep neural networks that were like actually deep.

Exploding gradient problem (Encountered in RNNs)

$\left|\frac{\partial L}{\partial w}\right|$ becomes really large. $(W^k_{ij})_{new}$ gets really different than $(W^k_{ij})_{old}$.

That'd be okay for maybe a few iterations, but we need convergence as we go through iterations.
There won't be convergence.

NO convergence

Why it occurs? Again, due to chain rule multiplication. Each values in the product become moderately large, the product becomes significantly larger. One such activation func" is

Bias-variance tradeoff

① As # of layers ↑↑ ⇒ we have more weights. There's a higher chance of overfitting.

of layers ↑↑ variance ↑↑ (overfitting)

② A 1-layer NN with a sigmoid neuron is just logistic regression (a linear separator).

of layers ↓↓ bias ↑↑ (underfitting)

Amount of fine tuning that can be done for say 10 wts is quite lesser than that for 1000 wts.

Typically MLPs face the problem of high variance, i.e. overfitting as opposed to underfitting.

Soln: MLP is an optimizn problem. Add regularizn.

$$L = \sum_{i=1}^{n} \text{loss}(y_i, \hat{y}_i) + \lambda \underbrace{\sum_{i,j,k} (W_{ij}^{k})^2}_{\text{L2 reg. on weights}}$$

λ ↑↑ overfitting ↓↓

L_1 regularizn creates sparsity for NNs as well. (weights become 0, meaning some edges are lost, creating a sparse network).

hyperparameters for a NN:

λ : weight of L_2 regularizer

& # of layers.